Assignment 1: Digital Logic and Combinational Circuits

Smart Parking Gate Controller

INSTRUCTIONS

1. Submission is only via Brightspace. Deadlines are strict.
2. The exercises in this assignment add up to 100 points. The passing score is 60/100. Points for each sub-question are indicated in square brackets.
3. Submit the following files in one ZIP archive named `StudentNumber_Assignment1.zip`:
   a) `report.pdf`: your typeset answers, using this LaTeX template;
   b) `parking_gate.circ`: your Logisim-evolution circuit;
   c) `README.txt`: your student number, Logisim-evolution version, and any known limitations.
4. All written answers must be typeset in LaTeX. Clearly labelled, legible diagrams may be created using diagrams.net, PowerPoint, TikZ, or another drawing tool and included as figures in the report. Handwritten solutions are not accepted.
5. **Logisim-evolution is the official and supported circuit simulator for this assignment.** Circuit screenshots do not replace the editable `.circ` file.
6. Use the signal names specified in the questions. This is necessary for consistent testing and grading.
7. Unless explicitly stated otherwise, you may use basic logic gates, wires, splitters, input/output pins, constants, multiplexers, decoders, and encoders. Do not use ROMs, RAMs, programmable logic components, or HDL components.
8. Equivalent Boolean expressions and logically equivalent circuit implementations are accepted, provided that the required functionality is satisfied.
9. You must show the main steps of your reasoning. Correct answers without sufficient derivation may receive partial credit.
10. Seeking solutions from the internet, from any external resource, or from another person is prohibited. University policies on plagiarism and academic integrity apply.
11. The course lecturer reserves the right to ask a student to explain any submitted answer or circuit. If the student cannot provide a satisfactory explanation, the corresponding question may receive partial or no credit.

In this assignment, you will apply the concepts from Chapters 1 and 2 of the course material to design and analyse parts of a simplified smart parking access system. The focus is on number representation, Boolean logic, combinational circuits, and standard combinational building blocks.

1. Number representation and encoding **(20 points)**

The parking garage can hold at most 15 vehicles. The current number of vehicles is represented as a 4-bit unsigned binary number $N = N_3N_2N_1N_0$.

(a) [4 points] Convert the following decimal values to 4-bit unsigned binary and hexadecimal:

$$0, \qquad 5, \qquad 9, \qquad 15.$$

(b) [4 points] Suppose $N = 1111_2$ and the value is incremented by one using a 4-bit unsigned adder. State the resulting 4-bit output and explain whether overflow occurs.

(c) [6 points] The bit pattern 1001_2 is interpreted in two different ways:

i) as a 4-bit unsigned number;

ii) as a 4-bit two's-complement number.

Give the corresponding decimal value in each case and explain why the same bit pattern can represent different values.

(d) [6 points] Convert the following values as indicated:

i) 10110110_2 to hexadecimal;

ii) $7A_{16}$ to binary;

iii) 45_{10} to 8-bit binary;

iv) -18_{10} to 8-bit two's-complement representation.

2. **Boolean logic and simplification** **(25 points)**

The access controller has the following one-bit inputs:

Signal	Meaning
`vehicle`	A vehicle is detected at the entrance
`card_valid`	The vehicle has a valid access card
`space_available`	At least one parking space is available
`emergency`	Emergency override is active

The one-bit output `permit` must satisfy the following requirements:

- In normal operation, entry is permitted only when a vehicle is present, its card is valid, and space is available.
- When `emergency`=1, entry is always permitted, regardless of the other inputs.

(a) [8 points] Construct the complete truth table for `permit`. Use the input order

(`emergency`, `vehicle`, `card_valid`, `space_available`).

(b) [4 points] Write the canonical sum-of-products expression for `permit`.

(c) [6 points] Simplify the expression using a four-variable Karnaugh map. Show all groups clearly.

(d) [2 points] Give the final simplified Boolean expression.

(e) [5 points] Implement the simplified circuit in the `PermissionLogic` subcircuit of Logisim-evolution. For this part, you may use only:

- AND, OR, and NOT gates;
- input and output pins;
- wires, constants, and splitters.

Use exactly the signal names listed above. Include a screenshot of the completed subcircuit in your report, and submit the editable `.circ` file.

3. **Combinational logic design** **(20 points)**

The controller also produces two indicator outputs:

- `green_light`=1 when a vehicle is present and entry is permitted;
- `red_light`=1 when a vehicle is present and entry is not permitted;
- both lights must be off when no vehicle is present.

(a) 4 points Derive simplified Boolean expressions for `green_light` and `red_light` in terms of `vehicle` and `permit`.

(b) 4 points Implement both outputs in the `IndicatorLogic` subcircuit in Logisim-evolution.

(c) 8 points Design a circuit with three inputs A, B, and C whose output F is 1 if and only if exactly two of the three inputs are 1.

i) Construct the truth table.
ii) Derive a Boolean expression.
iii) Simplify the expression if possible.

(d) 4 points Implement the exactly-two detector in a Logisim-evolution subcircuit named `ExactlyTwo` using only NOT gates and 2-input AND/OR gates.

4. **Multiplexers, decoders, and encoders** **(20 points)**

(a) 4 points A 2-to-1 multiplexer selects the access decision used by the system:

- when `mode`=0, the selected value is `card_valid`;
- when `mode`=1, the selected value is `manual_permit`.

The output is called `selected_permission`.

i) Write its Boolean expression.
ii) Draw a block-level multiplexer diagram.
iii) Implement it in the `PermissionSelect` subcircuit.

(b) 12 points A 2-to-4 decoder has inputs X_1X_0 and active-high outputs D_0, D_1, D_2, D_3.

i) Complete the decoder truth table.
ii) Write a Boolean expression for each output.
iii) Show how the decoder can be used to generate

$$F(X_1, X_0) = \Sigma m(1, 2, 3).$$

(c) 4 points A priority encoder has four request inputs R_3, R_2, R_1, R_0, where R_3 has the highest priority and R_0 the lowest. The output is a 2-bit code Y_1Y_0 identifying the highest-priority asserted request, together with a valid output V.

i) Complete the priority encoder truth table using "X (don't care)" entries where appropriate.
ii) Give the output code for each possible highest-priority request.
iii) Explain the purpose of the valid output V.

5. **Comparator design and propagation delay** **(15 points)**

Two unsigned 2-bit numbers are given by

$$A = A_1A_0, \qquad B = B_1B_0.$$

The circuit has two outputs:

$$G = 1 \text{ if } A > B, \qquad E = 1 \text{ if } A = B.$$

(a) 4 points Construct the complete truth table for G and E.

(b) 4 points Derive Boolean expressions for G and E. Simplify the expressions where possible.

(c) 4 points Implement the comparator in a Logisim-evolution subcircuit named `Comparator2bit`. Do not use the built-in comparator component.

(d) 3 points Assume each NOT gate has a propagation delay of 1 ns, and each 2-input AND or OR gate has a propagation delay of 2 ns. For your implementation of G, identify one critical input-to-output path and calculate its propagation delay.

Required functional tests

The following tests are provided to help you verify your designs. Additional hidden tests may be used during grading.

PermissionLogic tests

Test	`emergency`	`vehicle`	`card_valid`	`space_available`	Expected `permit`
C1	0	0	0	0	0
C2	0	1	0	1	0
C3	0	1	1	0	0
C4	0	1	1	1	1
C5	1	0	0	0	1
C6	1	1	0	0	1

ExactlyTwo tests

Test	A	B	C	Expected F
E1	0	0	0	0
E2	1	1	0	1
E3	1	0	1	1
E4	0	1	1	1
E5	1	1	1	0

Comparator2bit tests

Test	A	B	Expected G	Expected E
K1	00	00	0	1
K2	01	10	0	0
K3	10	01	1	0
K4	11	11	0	1
K5	11	10	1	0

Submission checklist

Before submitting, verify that:

- your ZIP archive has the required filename;
- your report is a readable PDF generated from this LaTeX template;
- all figures, truth tables, Karnaugh maps, and equations are labelled clearly;
- the Logisim file opens in Logisim-evolution;
- all required subcircuits and signal names are present;
- all published functional tests pass;
- your README identifies the Logisim-evolution version used.